

Comisión 3EK03

Ingeniería en sistemas de información

Año 2026

TRABAJO PRÁCTICO Nº 2

Nombre y apellido	Legajo	Firma
Agustin Gregoret	52285	
Leandro Carrion Lescano	50269	
Marco Subira	47880	

Profesores:

- Teoría: Díaz, Daniela.
- Práctica: Lombardo, Víctor.

Índice de Contenidos:

1.Enunciado…….....1
2.Resolución del problema de la mochila utilizando una Búsqueda Exhaustiva……..3
3.Resolución usando el Algoritmo Greedy y comparativa con el método Exhaustivo…….4
4.Planteamiento del problema de la mochila considerando Pesos (grs.) en lugar de Volumen.....6
5.Conclusion……...8

1. Enunciado

Perfilado de sección



Problema de la mochila

Consiste en elegir, de entre un conjunto de **n elementos**, (cada uno con un valor $i, y un volumen Vi), aquellos que puedan ser cargados en una mochila de volumen V de manera que el **valor obtenido** sea **máximo**.

Utilizando una computadora, resolver el siguiente problema:

Cuáles son los elementos de la lista siguiente que cargaremos en una mochila de 4200 cm3 de manera que su valor en $ sea máximo.

Objeto	Volumen (cm^3.)	Valor ($)
1	150	20
2	325	40
3	600	50
4	805	36
5	430	25
6	1200	64
7	770	54
8	60	18
9	930	46
10	353	28

Volumen máximo soportado por la mochila: 4200 cm^3.

Para su solución, utilizar un **procedimiento exhaustivo** que consiste en evaluar para cada subconjunto de elementos el valor correspondiente y, posteriormente, **clasificando los subconjuntos por su valor de mayor a menor**, encontrar cuál es el subconjunto solución.

Ejercicios:

1.Resolver el **problema de la mochila utilizando una búsqueda exhaustiva**.

2.Resolver el ejercicio anterior usando el **algoritmo greedy** y comentar su similitud o no con el exhaustivo.

3.Plantear el problema de la mochila teniendo en cuenta los **pesos** en lugar del volumen. Luego, **dados 3 elementos**, cuyos pesos son:

- 1800 grs.,
- 600 grs.
- y 1200 grs.

y cuyos valores son:

- $72,
- $36
- y $60 respectivamente, y con una mochila que puede soportar hasta **3000 grs.** se pide:

A) Hallar una solución **utilizando un algoritmo goloso y exhaustivo**.

B) **Analizar dicha solución respecto a su grado de optimización** y elaborar las **conclusiones** que considere adecuadas.

Desarrollo del Cuestionario

2. Resolución del problema de la mochila utilizando una Búsqueda Exhaustiva

La **Búsqueda Exhaustiva** evalúa el espacio completo de soluciones posibles. Para el conjunto inicial de 10 elementos, existen 2^{10}= 1024 combinaciones posibles. El algoritmo analiza matemáticamente cada una de ellas mediante combinaciones binarias, descarta las que superan la capacidad máxima permitida de la mochila (4200 cm³) y selecciona aquella combinación que maximice el valor total acumulado.

Para lograr esto, el programa utiliza la función exhaustiva basada en la librería itertools:

```python
def exhaustiva(capacidad, objetos):
    mejor_valor = 0
    mejor_combinacion = []

    # Genera todas las combinaciones binarias posibles (0 o 1) para los objetos
    combinaciones = product([0, 1], repeat=len(objetos))

    for combinacion in combinaciones:
        volumen_total = 0
        valor_total = 0
        combinacion_actual = []

        for j in range(len(objetos)):
            if combinacion[j] == 1:
                volumen_total += objetos[j][1]
                valor_total += objetos[j][2]
                combinacion_actual.append(objetos[j][0])

        # Actualiza el óptimo si la solución es válida y supera el valor previo
        if volumen_total <= capacidad and valor_total > mejor_valor:
            mejor_valor = valor_total
            mejor_combinacion = combinacion_actual

    return mejor_valor, mejor_combinacion
```

Resultado de la ejecución (10 objetos originales):

- **Capacidad de la mochila:** 4200 cm³
- **Mejor combinación encontrada:** `[1, 2, 3, 5, 6, 7, 8, 10]`
- **Volumen total utilizado:** 3888 cm³
- **Valor máximo obtenido:** $299

3. Resolución usando el Algoritmo Greedy y comparativa con el método Exhaustivo

El **Algoritmo Greedy (Goloso)** toma decisiones secuenciales basándose en un criterio de optimización local. En lugar de evaluar el espacio completo de soluciones, ordena los elementos bajo una prioridad fija y los introduce en la mochila hasta que esta se llene.

El comportamiento del algoritmo se define mediante la siguiente función

```python
def greedy(capacidad, objetos, opcion):
    # Definimos el criterio de ordenamiento según la opción seleccionada
    if opcion == "1":
        # Opción 1: Mayor Valor primero
        objetos_ordenados = sorted(objetos, key=lambda obj: obj[2], reverse=True
    elif opcion == "2":
        # Opción 2: Menor Volumen/Peso primero (de menor a mayor)
        objetos_ordenados = sorted(objetos, key=lambda obj: obj[1], reverse=Fals
    else:
        # Opción 3: Mejor relación Valor / Volumen (o Valor / Peso)
        objetos_ordenados = sorted(objetos, key=lambda obj: obj[2] / obj[1], rev

    combinacion = []
    volumen = 0
    valor = 0

    for objeto in objetos_ordenados:
        numero = objeto[0]
        volumen_objeto = objeto[1]
        valor_objeto = objeto[2]

        if volumen + volumen_objeto <= capacidad:
            combinacion.append(numero)
            volumen += volumen_objeto
            valor += valor_objeto

    return combinacion, volumen, valor
```

Al ejecutar el algoritmo Greedy con los 10 objetos originales (Capacidad: 4200 cm³), se obtienen los siguientes resultados comparativos contra la Búsqueda Exhaustiva:

Método / Criterio	Objetos Seleccionados	Volumen Utilizado (cm³)	Valor Total ($)	¿Es la solución Óptima?
Búsqueda Exhaustiva	`[1, 2, 3, 5, 6, 7, 8, 10]`	3888 cm³	**$299**	**Sí**
Greedy 1 (Mejor Valor)	`[6, 7, 3, 9, 2, 10]`	4178cm³	**$282**	**No**
Greedy 2 (Mejor Volumen)	`[8, 1, 2, 10, 5, 3, 7 , 4]`	3493 cm³	**$271**	**No**
Greedy 3 (Mejor Relación)	`[8, 1, 2, 3, 10, 7, 5, 6]`	3888 cm³	**$299**	**Sí**

Comentarios sobre similitud y comportamiento:

La búsqueda exhaustiva obtiene la solución óptima para la capacidad de 4200 cm³, seleccionando los objetos **[1, 2, 3, 5, 6, 7, 8, 10]**, con un volumen total de **3888 cm³** y un valor de **$299**.

Al comparar los métodos Greedy con la búsqueda exhaustiva, se observa que los resultados dependen del criterio utilizado para seleccionar los objetos.

El **Greedy 1, basado en el mayor valor individual**, obtiene un valor de **$282**, por lo que no alcanza la solución óptima. Esto demuestra que elegir siempre el objeto de mayor valor no garantiza obtener la mejor combinación global.

El **Greedy 2, basado en el menor volumen**, obtiene un valor de **$271**. Si bien permite utilizar la capacidad de manera relativamente eficiente, tampoco alcanza el valor máximo encontrado mediante la búsqueda exhaustiva.

Por último, el **Greedy 3, basado en la relación valor/volumen**, obtiene exactamente la misma combinación de objetos que la búsqueda exhaustiva y alcanza un valor de **$299**. En este caso particular, por lo tanto, el criterio de relación valor/volumen permite encontrar la solución óptima.

Sin embargo, este resultado no significa que el algoritmo Greedy garantice siempre una solución óptima. La búsqueda exhaustiva evalúa todas las combinaciones posibles, mientras que Greedy toma decisiones de manera local siguiendo el criterio seleccionado. Por este

motivo, un algoritmo Greedy puede obtener una solución óptima en determinados casos, pero también puede producir soluciones subóptimas en otros.

Diferencia de Rendimiento:

La principal diferencia entre ambos enfoques se encuentra en la cantidad de combinaciones que deben analizar. La búsqueda exhaustiva evalúa 2^n combinaciones para un problema con n objetos, por lo que el número de posibilidades crece exponencialmente a medida que aumenta la cantidad de objetos. En cambio, el algoritmo Greedy no evalúa todas las combinaciones, sino que ordena los objetos según un determinado criterio y los recorre seleccionando aquellos que pueden incorporarse a la mochila. Por este motivo, Greedy presenta un menor costo computacional, aunque no garantiza obtener siempre la solución óptima.

4. Planteamiento del problema de la mochila considerando Pesos (grs.) en lugar de Volumen

Para este escenario, se evalúa una nueva colección reducida de 3 elementos con una capacidad máxima soportada de **3000 grs.**

A nivel de software, el programa reutiliza de forma limpia las funciones genéricas de optimización pasándole el conjunto de datos configurado con pesos en la función comparar()

```python
def comparar():
    # Lista basada estrictamente en Pesos (grs.) y Valores ($)
    objetos2 = [
        (1, 1800, 72),  # (nroElemento, Peso, Valor)
        (2, 600, 36),
        (3, 1200, 60)
    ]

    # ... código de interacción de menús ...

    # Ejecución dinámica del Algoritmo Greedy pasando el criterio seleccionado
    combinacion, peso_greedy, valor = greedy(3000, objetos2, opc)

    # Ejecución de la Búsqueda Exhaustiva sobre el nuevo conjunto
    mejor_valor, mejor_combinacion = exhaustiva(3000, objetos2)

    # Cálculo del peso exacto acumulado por el método exhaustivo
    peso_exh = sum(obj[1] for obj in objetos2 if obj[0] in mejor_combinacion)
```

A) Soluciones obtenidas por ambos algoritmos (Utilizando el Criterio 3: Relación Valor/Peso)

I. Resolución por Búsqueda Exhaustiva:

El algoritmo evalúa los 8 subconjuntos posibles (2^3). La combinación matemática que maximiza la ganancia económica sin superar los 3000 grs. es la unión del **Elemento 1 y el Elemento 3**.

- **Elementos seleccionados:** `[1, 3]`
- **Peso total utilizado:** 1800 grs. + 1200 grs. = 3000 grs.
- **Valor máximo obtenido:** $72 + $60 = **$132**

II. Resolución por Algoritmo Greedy (Criterio 3: Relación Valor/Peso):

1. Se calculan las densidades de valor y se ordena de mayor a menor:

   Elemento 2 ($36/600g = 0.060) →Elemento 3 ($60/1200g = 0.050) → Elemento 1 ($72/1800g = 0.040).

2. Inserta el **Elemento 2** (600 grs., restan 2400 grs. de capacidad).
3. Inserta el **Elemento 3** (1200 grs., restan 1200 grs. de capacidad).

4. Evalúa el **Elemento 1** (1800 grs.), pero al superar el espacio remanente de la mochila, queda excluido.

- **Elementos seleccionados:** `[2, 3]`
- **Peso total utilizado:** 600 grs. + 1200 grs. = 1800 grs.
- **Valor total alcanzado:** $36 + $60 = **$96**

B) Análisis de optimización y conclusiones

Algoritmo	Combinación Resultante (IDs)	Peso Total Utilizado (grs.)	Valor Total Alcanzado ($)	Grado de Optimización
Búsqueda Exhaustiva	`[1, 3]`	3000 gramos	**$132**	**100% (Óptimo Absoluto)**
Algoritmo Greedy	`[2, 3]`	1800 gramos	**$96**	**72.7% (Subóptimo)**

5.Conclusiones:

1. **Invalidez del óptimo local:** En esta instancia basada en pesos, **el Algoritmo Greedy falló en hallar la solución óptima**. Mientras que la fuerza bruta llenó la mochila a su capacidad exacta (3000 grs.) logrando $132, la heurística golosa se estancó en $96, dejando un espacio ocioso de 1200 gramos que no pudo ser aprovechado.
2. **Demostración de la "Miopía" Greedy:** El algoritmo seleccionó en primera instancia el Elemento 2 porque ofrecía el mejor rendimiento inmediato por unidad de peso (0.06). Sin embargo, esta decisión local e irreversible bloqueó estructuralmente el espacio, imposibilitando la posterior combinación de los elementos 1 y 3 que, de manera conjunta, representaban una alternativa mucho más rentable para el sistema global.
3. **Lección teórica fundamental:** El problema de la mochila demuestra que ningún método es perfecto por sí solo. El algoritmo Greedy (Goloso) es una herramienta excelente porque es ultra rápido y da soluciones muy buenas en milisegundos, lo cual es ideal si tuviéramos miles de objetos. Sin embargo, como vimos en el ejercicio de los pesos, tiene el defecto de ser "miope" y no garantiza el éxito absoluto. Si el problema exige encontrar obligatoriamente la mejor combinación posible al 100%, la Búsqueda Exhaustiva es el único camino seguro, sabiendo que solo sirve para listas de objetos muy cortas debido al alto costo de procesamiento que requiere.